KV CACHE – NOTEBOOK NOTES

Inference basics • Memory calculation • MHA vs MQA

Purpose: A shortened, handwriting-friendly version of the 32-page class notes. It keeps the core concepts, formulas, worked example and important distinctions. Detailed class Q&A, admin discussion and repeated explanations have been removed.

1. Transformer foundations

Tokens, embeddings and Q–K–V

- Text is split into tokens. Each token is mapped to an integer token ID.
- An embedding table converts each token ID into a vector.
- Each token produces three vectors: Query (Q), Key (K) and Value (V).

Vector	Intuition
Query (Q)	What am I looking for?
Key (K)	What information do I offer / how can I be found?
Value (V)	What information do I contribute?

Scaled dot-product attention:
Attention(Q,K,V) = softmax(QK■ / √d■) V
Dividing by √d■ controls the scale of scores before softmax.

Causal mask

- A decoder token may attend to itself and earlier tokens, but never future tokens.
- Future-position scores are set to −∞ before softmax; their attention weights become zero.
- This is what makes generation autoregressive.

Autoregressive generation

- Generate one token → append it to the sequence → run the next step → repeat.
- The model uses the last position to predict the next token. Generation stops at the configured maximum-token limit or a stop token.

2. Naive decoding: why caching is needed

Autoregressive property: each new token attends to all earlier tokens. Without a cache, the model recomputes old tokens' K and V at every decode step.

Repeated work without a cache

Step	Sequence length	K/V recomputed
1	4	4 tokens
2	5	5 tokens (first 4 repeated)
3	6	6 tokens (first 5 repeated)
4	7	7 tokens

- Only the newest token has new K and V; all earlier K/V values are unchanged.
- For n decode steps, total work is 1 + 2 + … + n.

Naive decoding work = $n(n + 1) / 2 \rightarrow O(n^2)$
The repeated computation grows quadratically with sequence length.

The KV cache: the solution

- Store the K and V vectors of already processed tokens, for every transformer layer.
- At the next step, calculate K and V only for the new token and append them to the cache.
- The new token's Query attends to the cached K and V from the full previous sequence.
- Only K and V are cached — not Q and not the attention output.
- The cache is used during inference and normally resides in GPU VRAM.

Remember: KV cache trades extra memory usage for less repeated computation. With caching, each step adds only the new token's K/V rather than rebuilding all previous K/V.

Where it fits

- The model weights are loaded onto the GPU first. The KV cache occupies additional VRAM.
- The same principle applies wherever attention K/V are reused, including relevant encoder–decoder attention paths.

3. Inference: prefill and decode

Phase	What happens	Main latency measure
Prefill	Process the entire prompt; calculate and store K/V for prompt tokens. Parallel across positions.	TTFT – time to first token
Decode	Generate output tokens one at a time; extend the cache at each step.	TPOT – time per output token

- TTFT: time from request submission until the first output token appears.
- TPOT: time between successive generated tokens. TPS means tokens per second.
- Long prompts increase prefill work and can increase TTFT. Long answers require many sequential decode steps.
- Streaming displays tokens as they are generated, so users do not wait for the complete answer.

Two request patterns

Request	Prompt	Output	Likely bottleneck
A	10,000 tokens	20 tokens	Prefill / TTFT
B	20 tokens	10,000 tokens	Decode / TPOT

4. The memory wall

- GPU compute throughput describes how many operations the GPU can perform per second (often measured in TFLOPS).
- Memory bandwidth describes how quickly data can be transferred to the GPU.
- Arithmetic intensity = computation performed per byte moved from memory.

Memory wall: the GPU can compute faster than memory can supply data. It waits for data, so its compute capacity is underused.

- Analogy: a chef can cook 100 dishes per second, but the kitchen supplies ingredients for only 20. Ingredient supply is the bottleneck.
- Decode is especially affected because it repeatedly reads model weights and the growing K/V cache.
- Reducing K/V data to move can improve memory use and decode speed.

5. KV cache memory calculation

Formula to remember
KV bytes per token = 2 × num_KV_heads × head_dim × num_layers × dtype_bytes

Total KV cache = KV bytes per token × sequence length × batch size

- Factor 2 accounts for both Key and Value.
- Q is not included because Q is not stored in the KV cache.

Data type	Bytes per value
FP16 / BF16	2
FP32	4
FP64	8

Worked example from the whiteboard

- KV heads = 8; head dimension = 64; layers = 32; FP16 = 2 bytes.

Step	Calculation	Result
K per layer	8 × 64	512 values
V per layer	8 × 64	512 values
K + V	512 + 512	1,024 values
Bytes / token / layer	1,024 × 2 bytes	2,048 bytes = 2 KB
All 32 layers	2 KB × 32	64 KB / token
10,000 tokens	64 KB × 10,000	≈ 640 MB
100,000 tokens	64 KB × 100,000	≈ 6.4 GB

What makes the cache larger?

- More layers → larger cache.
- More KV heads → larger cache.
- Longer sequence length → larger cache, linearly.
- Larger batch size → larger cache, linearly. Sequence length and batch size multiply.
- FP32 uses twice the memory of FP16 for the same cache.

Main design lever: reduce the number of KV heads (as in MQA or GQA). Quantisation can reduce dtype bytes, but the tutor described it as a later option.

6. Multi-Head Attention (MHA)

- The model dimension is split across multiple attention heads.
- Each head has its own Q, K and V projections, performs attention, and produces an output.
- Head outputs are concatenated and passed through an output projection.
- Example: d_model = 512 and 8 heads → head_dim = 512 / 8 = 64.

In MHA with 8 heads, each token produces 8 Q heads, 8 K heads and 8 V heads. The cache stores K and V for all 8 heads.

7. Multi-Query Attention (MQA)

Core idea: keep multiple Query heads, but share one Key head and one Value head across all of them.

Feature	MHA (8 heads)	MQA (8 query heads)
Query heads	8	8
K heads	8	1 shared
V heads	8	1 shared
Q projection	512	512 (unchanged)
K projection	512	64
V projection	512	64
KV cache	Larger	Smaller

- MQA changes the K/V projection dimensions and the number of KV heads — it does not reduce the number of Query heads.
- Each Query head remains different, so attention scores can still differ even though K and V are shared.
- With 32 query heads and 1 shared KV head, the K/V cache can be 32× smaller than MHA.

Benefits and trade-off

- Benefits: smaller cache, less K/V data movement, faster decoding and more room for concurrent requests.
- Cost: fewer distinct K/V representations can reduce attention diversity and slightly affect quality.
- MQA must be part of model training. An MHA-trained model cannot simply be switched to MQA after training.

GQA (Grouped-Query Attention) is the middle ground between MHA and MQA: it shares K/V within groups rather than across every query head.

8. What the MQA paper demonstrated

Shazeer (2019), “Fast Transformer Decoding: One Write-Head is All You Need.”

- The paper identifies memory bandwidth as a major bottleneck during incremental decoding.
- MQA shares K and V across attention heads to reduce memory traffic.

Measure	MHA	MQA
WMT14 BLEU (dev)	26.7	26.5
Decoder time / output token	46 µs	3.8 µs
Billion-Word dev perplexity	29.9	30.2

- The reported quality difference was small in these experiments, while decoding became substantially faster.
- These are results from the paper’s specific experimental setup, not a guarantee for every model or workload.

9. Class demo: with vs without cache

- The notebook generated 300 tokens using SmolLM2-1.7B and compared use_cache=False with use_cache=True.

Run	Average time (saved notebook)
Without cache	17.617 s
With cache	8.171 s
Observed speed-up	≈ 2.16×

- The class run and saved notebook run had different timings, but both showed that caching roughly halved generation time in that demo.
- With greedy decoding, the generated text was the same; caching changed speed, not the intended output.

10. Practical points to remember

- In Hugging Face Transformers, use_cache=True is normally the default; the demo disabled it only for comparison.
- Each request has its own cache. Cache retention or eviction depends on available VRAM and request/context sizes.
- KV cache is different from conversation history stored in systems such as Redis.
- Prompt caching and KV caching are related ideas but are not the same cache.

11. Final revision sheet

Must-remember facts

- Autoregressive decoding = one token at a time.
- Naive decoding repeats old K/V calculations: $1 + 2 + \ldots + n = n(n + 1)/2 = O(n^2)$.
- KV cache stores K and V only, per layer, in GPU memory.
- Prefill processes the prompt and builds the cache; decode generates tokens sequentially.
- TTFT is associated with prefill; TPOT is associated with decode.
- KV cache size = 2 × KV heads × head_dim × layers × bytes × sequence length × batch size.
- MHA: each head has its own Q/K/V. MQA: multiple Q heads share one K/V pair.
- MQA reduces KV memory, but can reduce attention diversity. It must be trained into the model.

Quick glossary

Term	Meaning
Token / embedding	Token is a text unit; embedding is its vector representation.
Causal mask	Prevents a position from attending to future positions.
Inference	Using a trained model to make predictions.
Prefill / decode	Prompt processing / token-by-token generation.
TTFT / TPOT / TPS	Time to first token / time per output token / tokens per second.
VRAM / RAM	GPU memory / system memory.
MHA / MQA / GQA	Multi-Head / Multi-Query / Grouped-Query Attention.
Quantisation	Representing values with lower numerical precision.

Check yourself

- Why does naive decoding have quadratic repeated work?
- Why are K and V cached but Q is not?
- What is the difference between prefill and decode?
- Write the KV cache memory formula from memory.
- What changes in MQA, and what stays the same?
- What is the main benefit and the main trade-off of MQA?

Source basis: uploaded “KV Cache – 1” class notes (32 pages), including the whiteboard calculations, notebooks, demo and Shazeer (2019) paper summary. This version condenses the source; it does not add outside material.